

Collaboration Space Research & Requirements

A meeting-ready proposal based on the team's actual discussion, Dolly's real-time workflow, and official engineering collaboration practices.

Prepared by Mr. Nero / 14 July 2026

Executive recommendation

Build a Dolly Control Room, not another GitHub, MLflow, or Grafana.

The custom space should be the team's coordination layer: it shows ownership, dependencies, workload, decisions, experiments, integration health, and demo readiness. Code remains in GitHub; detailed ML runs can remain in MLflow or a lightweight equivalent; live metrics/logs can remain in a telemetry system. The workspace links or summarizes them in one place.

What the team actually discussed

Team statement	Meaning for the workspace
Ceaser is building a custom team collaboration space.	The design can include Dolly-specific features, not only generic project management.
Send one document at a time.	There must be one canonical artifact with version, owner, status, and a clearly marked latest copy.
The workflow was refined around skills, scalability, and sustainability.	Ownership, interfaces, dependencies, and future changes must be visible in the same model.
Jeboy has fewer colored blocks, but each block contains a huge amount of work.	Workload cannot be measured by task count alone. Track effort, risk, review load, and acceptance evidence.
The real-time function is very difficult.	The space needs test runs, latency/FPS/confidence metrics, interface contracts, failure logs, and replayable evidence.
Each member should prepare a feature list for the 22:00 meeting.	Tonight's result should be a small P0 scope, owners, and the first checkpoint - not an unlimited feature wishlist.

The central design problem

Dolly combines multi-camera capture, synchronization, perception, speaker/subject understanding, shot selection, camera control, switching, recording, UI, and storytelling. A failure in one stage can block several later stages. Therefore, the workspace must organize work by pipeline stage and show integration gates between stages.

1. Requirements derived from Dolly and the team

Observed need	Required feature	Minimum behavior
Three different strengths	Role-based views	Ceaser: AI/ML; Nero: integration/fullstack; Jeboy: product/demo. Shared view remains the source of truth.
12-hour time difference with Montréal	Async check-in + handoff	Done / Next / Blocked / Need help, with timestamp and the next person expected to act.
Ceaser may not always access Discord	Workspace announcements	Important decisions and deadlines must live in the workspace; Discord is a notification channel, not the archive.
Large dependency graph	Pipeline board + blockers	Blocked by / Blocking, interface owner, expected input/output, and integration test state.
Unequal task sizes	Effort and risk fields	S / M / L / XL effort, technical risk, review owner, and acceptance evidence. Never compare contribution using block count.
Fast hackathon MVP	P0 scope guard	Every feature/task must connect to MVP, demo, or risk reduction. Everything else moves to Later.
Real-time AI pipeline	Experiment + telemetry records	FPS, end-to-end latency, confidence, failure reason, test clip, code version, and chosen baseline.
Live demo risk	Demo room	Runbook, checklist, device state, fallback mode, known issues, final assets, and rehearsal result.
Many research/files	Canonical artifact registry	Title, type, owner, version, status, linked task/decision, and latest/archived marker.

Research-backed principles

PLAN AS VIEWS, NOT DUPLICATES

- GitHub Projects already supports table, board, roadmap, custom fields, issues, and pull requests [1].
- The custom space should surface this state instead of creating a second disconnected task truth.

TRACK EVIDENCE, NOT CLAIMS

- MLflow's model is parameters + metrics + artifacts + comparable runs [3].
- Dolly experiments should always link a metric and a test artifact.

OBSERVE THE WHOLE PIPELINE

- OpenTelemetry groups traces, metrics, and logs as complementary signals [5].
- A Dolly run needs end-to-end timing plus stage-specific failures.

2. Recommended Dolly Control Room

View	What it answers	Key content
COMMAND CENTER	Are we on track today?	Countdown, P0 progress, current blockers, owners needing action, latest decisions, next integration test.
PIPELINE	Where is Dolly blocked?	Capture & Sync -> Perception -> Decision -> Control -> Output. Each stage shows owner and integration gate.
TASKS & ROADMAP	Who does what, by when?	Board, timeline, dependencies, effort, risk, review state, and GitHub link.
DOLLY LAB	Which approach actually works?	Experiment runs, test clips, parameters, accuracy, confidence, FPS, latency, failure notes, selected baseline.
INTERFACES	Will components connect?	Input/output contract, endpoint/event, sample payload, owner, version, test status, and known incompatibilities.
DEMO ROOM	Can we show this safely?	Device checklist, runbook, fallback path, rehearsal result, known issues, pitch assets, latest build.
DOCS & DECISIONS	What is the latest truth?	Architecture, setup, research, canonical artifacts, meeting notes, decisions, reasons, and linked tasks.

Core objects and required fields

Object	Required fields	Definition of complete
Task	Owner, stage, priority, effort, risk, due date, blocker/dependency, reviewer, GitHub link.	Acceptance condition met; evidence attached; dependency state updated.
Experiment	Goal, code/data version, input clip, parameters, metrics, artifact, result, next step.	Comparable result exists and the team can reproduce or reject it.
Interface	Producer, consumer, payload/schema, timing, version, sample, owner, test status.	Both sides pass the same integration test.
Decision	Question, decision, reason, owner, date, affected tasks, replacement decision.	The team knows what changed and no longer debates the old choice without new evidence.
Demo run	Build, devices, scenario, start/end, pass/fail, failure stage, video/log, fallback result.	A repeatable run succeeds under the final setup.
Artifact	Type, owner, version, status, latest flag, task/decision link, file/link.	Only one item is marked current for the same purpose.

Important workload rule

The dashboard must show effort, risk, blockers, and review load - not only the number of assigned blocks. This directly answers Ceaser's point that Jeboy may own fewer blocks while each block contains substantial product, design, narrative, and demo work.

3. Priority: what to build now vs later

Phase	Features	Why now
P0 - FIRST	Command Center; pipeline board; tasks/roadmap; owner/deadline/effort/risk; blocker/dependency; canonical files; decision log; async check-in; demo checklist.	These remove coordination risk immediately and use the exact problems raised in the chat.
P1 - DOLLY BUILD	GitHub sync; experiment tracker; interface registry; integration test status; role-based views; demo run records.	These connect technical evidence to work as the real-time MVP starts running.
P2 - AFTER DATA	Live telemetry dashboard; traces/logs; alerts; automated summaries; workload analytics; advanced search.	Useful only after the pipeline emits reliable data. Do not spend early tokens/time on empty dashboards.

Build vs integrate

System	Owns	Dolly workspace should
Custom workspace	Coordination, decisions, workload, interface status, demo readiness.	Be the home screen and source of project truth.
GitHub	Issues, code, branches, pull requests, reviews, milestones.	Link/sync items; avoid duplicating code-review state. GitHub supports projects, fields, roadmaps, dependencies and PR review [1][2][6].
MLflow or lightweight equivalent	Experiment parameters, metrics, artifacts, comparable runs.	Show selected metrics and deep-link to evidence; start lightweight if MLflow setup is too expensive [3].
Grafana/telemetry UI	At-a-glance metrics/logs and alerting.	Embed/link panels only when Dolly emits stable telemetry [4].
Discord/email	Fast notification and meeting coordination.	Send reminders and mentions; keep durable decisions in the workspace.

Suggested implementation sequence

STEP 1

- Define objects and fields
- Import the current workflow
- Agree on P0 and owners

STEP 2

- Create Command Center
- Create pipeline/task views
- Add canonical artifacts

STEP 3

- Connect GitHub
- Add experiments/interfaces
- Record first demo run

4. Team-specific views (proposed - confirm tonight)

Member	Primary view	What must be visible	Handoff
CeaserZhao	AI / Perception / Experiments	Dataset/test clip, model or method, confidence/accuracy, FPS/latency, artifacts, selected baseline, integration contract.	Output schema + timing + known failure cases to Nero.
Mr. Nero	Integration / Backend / Control	API/events, component health, pipeline latency, camera/control connection, frontend state, deployment, error logs.	Working build + telemetry + demo controls to Jeboy.
Jeboy	Product / UI / Story / Demo	User flow, shot behavior, visual quality, narrative rules, demo scenario, pitch assets, rehearsal feedback.	Acceptance criteria and demo evidence back to the team.
Shared	Command Center	P0 progress, blockers, decisions, next integration gate, latest build, demo readiness.	Daily async update and next owner.

5. Questions that must be answered in the meeting

01	Which P0 features are already present in Ceaser's space, and which are missing?
02	Will GitHub be the code/task source of truth, or only the code source of truth?
03	What are the confirmed pipeline stages and owners from the current diagram?
04	What fields define task effort, risk, blocker, reviewer, and completion evidence?
05	What interface must each stage publish to the next stage?
06	Which metrics define a usable real-time baseline: FPS, end-to-end latency, confidence, switching delay, or stability?
07	How will the team record a failed experiment or failed demo run so it is still useful?
08	What is the first workspace checkpoint, owner, and deadline?

Recommended meeting outcome

- Leave the call with exactly four things:
- A confirmed P0 list.
 - One owner and one deadline for every P0 item.
 - A confirmed Dolly pipeline/role map.
 - A date for the first workspace and integration checkpoint.

6. Message Mr. Nero can send to the team

I researched this based on our Dolly workflow and what we discussed today. I think our space should work like a Dolly Control Room, not just a normal task manager.

My main feature list is:

1. A Command Center for milestones, blockers, decisions, and demo status.
2. A pipeline view for capture, perception, decision, control, and output stages.
3. Tasks with owner, deadline, effort, risk, dependencies, reviewer, and completion evidence.
4. A canonical file and document system, so we always know the latest version.
5. Daily async updates and clear handoffs because of our time difference.
6. A Dolly Lab for experiments, test videos, confidence, FPS, latency, results, and failures.
7. An interface page showing how the AI, backend, cameras, UI, and control parts connect.
8. GitHub integration for issues, PRs, commits, and reviews.
9. A Demo Room with a runbook, checklist, fallback mode, known issues, pitch files, and rehearsal results.

Also, I think we should track effort and risk, not only task count. Like u said, Jeboy may have fewer blocks, but each one can still be a huge amount of work.

For now, we should only choose the P0 features and avoid building too much. Tonight we can check what already exists, what we need to add, who owns each part, and when the first version should be ready.

7. Sources and evidence

[1]	GitHub Docs - About Projects	Projects supports table, board, roadmap, custom fields, status updates, automation, and integration with issues and pull requests.	Open source
[2]	GitHub Docs - Issue dependencies	Dependencies show which issues are blocked by or blocking other work.	Open source
[3]	MLflow - Experiment Tracking	Tracking records parameters, metrics, metadata, and artifacts; runs can be compared and reproduced.	Open source
[4]	Grafana - Dashboards and Alerting	Dashboards provide an at-a-glance view; alerting watches metrics/logs for important conditions.	Dashboard / Alerting
[5]	OpenTelemetry - Signals	Traces, metrics, and logs provide different views into system behavior and end-to-end execution.	Open source
[6]	GitHub Docs - Pull request reviews	Reviews allow discussion, suggested improvements, approval, or change requests before merge.	Open source

Research interpretation: the official sources describe proven capabilities of engineering tools. The Dolly-specific layout, priorities, workload rule, role views, and meeting requirements are the report author's synthesis from the team's chat, current workflow diagram, project roles, and hackathon constraints.

Ceaser shared this diagram after carefully reading the Dolly strategy and MVP report, then refining the workflow around team skills, scalability, and sustainability. It is preserved as the current visual reference; its content has not been redrawn or changed.

